

Canonical Structural Analysis: A Deterministic Compilation Framework for Workload-Relative Dependency Analysis

Jose Luna

July 6, 2026

Abstract

Paper 1 formalized dependency as a workload-relative property computed on a directed graph by deletion and reachability. This paper addresses the prior compilation question: what exactly is the graph on which that predicate operates? We define canonical structural analysis as a deterministic compilation framework that maps heterogeneous system descriptions into canonical typed structural objects. The framework separates domain extraction, normalization, canonicalization, workload projection, and reachability-graph construction. Rather than identifying any existing representation as universal, the paper treats compiler IRs, dependence graphs, provenance models, workflow graphs, and canonical graph representations as ingredients in a broader structural compilation model. The central object is a canonical structural object from which workload-specific directed graphs can be projected for dependency analysis. The contribution is not a new graph theory or a replacement for compiler theory. It is a formal interface between concrete systems and workload-relative dependency predicates, designed to make analysis deterministic, representation-independent, and suitable for later analyses such as dependency, resilience, redundancy, and governance.

1 Introduction

Paper 1 defined a workload-relative dependency predicate on a directed graph, evaluated by counterfactual deletion and reachability. It did not define how that graph is obtained from a concrete system. This paper fills that gap.

The central question is:

What exactly is G ?

We answer by defining a deterministic *structural* pipeline that maps heterogeneous system descriptions into a canonical structural object, and then projects that object into the workload-specific graph required by the dependency predicate.

Figure 1 previews the end-to-end interface: concrete systems enter on the left, and Paper 1’s dependency analysis consumes the projected reachability graph on the right.

A missing bridge (Paper 1 vs. this paper). Paper 1 assumes the existence of an analysis graph G on which deletion and reachability are evaluated. The purpose of this paper is to construct that object: given a concrete system description d , we define a deterministic compilation function C_χ that produces the workload-class graph G_χ (and the induced workload anchors) as an explicit, representation-independent artifact.

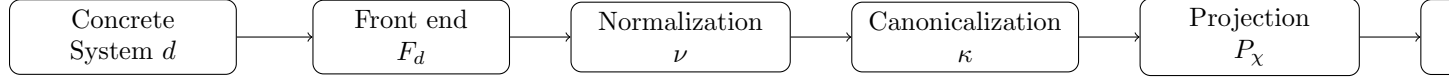


Figure 1: Overall compilation pipeline connecting concrete systems to the workload-relative dependency predicate of Paper 1.

2 Motivation

Problem. Heterogeneous systems are increasingly analyzed with graph-based tools, but each domain exposes different structural artifacts: compiler IRs and program graphs for software [10, 11], workflow and provenance graphs for data systems [7, 5], and typed knowledge graphs for organizational and socio-technical structure [8]. As a result, “the graph” used in downstream dependency reasoning is often implicit, ad hoc, or tightly coupled to a particular representation.

Gap. Existing IRs, code property graphs, provenance models, and workflow graphs preserve useful structure [10, 11, 15, 7, 5], but none defines a deterministic *cross-domain compilation contract* into the exact directed graph required by workload-relative dependency analysis. In particular, representation artifacts (naming, ordering, encoding choices) can leak into the analysis unless canonicalization and workload-class projection are made explicit.

Framework. As illustrated in Figure 1, Canonical Structural Analysis defines a deterministic structural compiler

$$C_\chi = U \circ P_\chi \circ \kappa \circ \nu \circ F_d$$

as the pipeline from a concrete system description to a workload-specific reachability graph. Extraction F_d and normalization ν recover domain structure; canonicalization κ erases representation artifacts; projection P_χ selects the information required by a workload class χ ; and U constructs the directed graph consumed by the dependency predicate of Paper 1.

Contributions.

- (1) Defines the canonical structural object Σ .
- (2) Defines a deterministic compilation function C_χ .
- (3) Proves determinism, idempotence, projection closure, representation invariance, workload preservation, compiler correctness, and minimal sufficiency.
- (4) Separates compilation cost from query cost.

3 Related Work

This paper is positioned between compiler intermediate representations, program analysis graphs, code property graphs, provenance/workflow models, and canonical graph representations. Rather than proposing a new IR, we treat these bodies of work as established sources of structural information and as motivation for a compilation contract that (i) is deterministic, (ii) explicitly removes representation artifacts via canonicalization, and (iii) produces workload-class projections tailored to the dependency predicate of Paper 1.

3.1 Compiler intermediate representations (LLVM, MLIR, IR design)

Compiler intermediate representations provide a long-standing template for separating front-end extraction from optimization and analysis. LLVM IR is a widely used low-level IR with a stable specification and an ecosystem of analyses and transforms [10]. More broadly, IR design emphasizes invariants, well-defined transformations, and analysis-friendly forms [13].

MLIR extends this lineage by making heterogeneity explicit: multiple dialects can coexist, and lowering can proceed in stages while preserving enough structure for progressively different analyses [11]. This staged approach aligns with our separation of extraction and normalization from subsequent canonicalization and projection.

Stage mapping. In the language of the structural compiler, LLVM/MLIR primarily inform F_d (how to extract structure from concrete artifacts) and ν (how to normalize into analysis-friendly forms). They can also provide concrete substrates for H when the domain is software, but they do not fix a domain-independent κ or a workload-class projection contract P_χ .

Contribution and gap relative to this framework. IRs (LLVM/MLIR) contribute a principled pipeline view and concrete realizations of F_d and ν . They do not, by themselves, define a canonical representative across representation-equivalent compilations, nor do they make workload-relative projection and representation invariance explicit as part of the analysis interface. Our framework is designed to be *implemented using* existing IR infrastructures when the source domain is software, while remaining applicable to non-compiler domains.

3.2 Program analysis graphs (CFG, SSA, dominators, PDG, slicing)

Graph representations are foundational in program analysis. Control-flow graphs (CFGs) and dominator trees support reachability and dominance reasoning in flowgraphs [9]. Static single assignment (SSA) form provides an analysis-friendly representation of def-use structure and enables many classic optimizations [4]. Dependence graphs, and in particular the program dependence graph (PDG), make control and data dependences explicit and support optimizations and program understanding [6]. Program slicing further operationalizes dependency-like questions by computing the subset of program elements relevant to a slicing criterion [14].

These representations highlight two themes that motivate our pipeline. First, multiple graph views can be extracted from a single underlying artifact, depending on the analysis goal. Second, the correctness of downstream reasoning depends on the invariants guaranteed by earlier compilation stages.

Stage mapping. These analyses contribute most directly to P_χ and U : they motivate the idea that a workload class should induce a projection to a particular relation family (e.g., control, data, dominance) and that many queries reduce to reachability computations on an analysis graph. In our terms, CFG/SSA/PDG structure can either live inside H as typed edges or be produced by P_χ from a richer H , after which U constructs the directed reachability graph consumed by Paper 1.

Contribution and gap relative to this framework. Program analysis graphs contribute well-studied edge semantics and a vocabulary for workload-driven projection. They typically assume a code-centric source domain and do not provide a representation-invariance layer for comparing dependency answers across alternative front ends, optimization pipelines, or non-program systems. Our framework retains their graph-theoretic strengths while making κ , P_χ , and U explicit components of a deterministic compilation contract.

3.3 Code property graphs and security-oriented unified representations

Code property graphs (CPGs) unify syntactic (AST), control-flow, and dependence information into a single, queryable graph that supports scalable vulnerability discovery [15]. The CPG line of work demonstrates that unifying multiple relation families can improve analysis ergonomics, especially when paired with graph query languages and domain-specific traversals.

Stage mapping. CPGs contribute most naturally to the *structural graph model* H : they exemplify a typed attributed graph that integrates multiple relation families. Operationally, CPG construction can be viewed as part of F_d (extraction) and ν (normalization) for software domains, while vulnerability queries correspond to workload-like projections P_χ and reachability-style computations after U .

Contribution and gap relative to this framework. CPGs contribute a practical demonstration of multi-relation unification and workload-like querying in security contexts. They do not directly address how to compile heterogeneous *non-code* systems into a canonical structural object, nor do they formalize a canonicalization step κ that factors out representation artifacts across multiple upstream encodings. In our setting, a CPG is best viewed as one possible realization of H for software, with κ and P_χ making the representation- and workload-interface explicit.

3.4 Provenance models, workflow graphs, and knowledge graph structure

Provenance and workflow models provide canonical examples of non-compiler domains in which dependency-like questions are central. The W3C PROV family standardizes entities, activities, agents, and derivation/usage relations [7]. In database and data-management communities, provenance has also been formalized as a mechanism for explaining outputs in terms of inputs and transformations [2, 3]. Scientific workflow systems and DAG-based schedulers similarly treat computation as a graph of tasks and artifacts, supporting reproducibility, debugging, and what-if reasoning [5].

Knowledge graph research (RDF-style graphs, property graphs, and schema/ontology layers) further motivates typed attributed graph models in which vertices and edges carry semantic types and metadata [8].

Stage mapping. These literatures contribute primarily to H (a typed, attributed structural representation) and, implicitly, to $\mathcal{W}$ (workload spaces expressed as explanation, lineage, and what-if questions). Concretely, many provenance/workflow systems already materialize graphs that can serve as H after normalization ν ; our framework adds an explicit κ to remove representation artifacts and a workload-class projection P_χ that targets the specific reachability graph required by the dependency predicate.

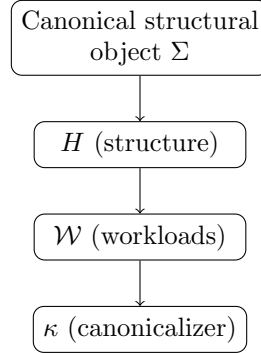
Contribution and gap relative to this framework. Provenance/workflow/knowledge-graph literatures contribute rich typed relation vocabularies outside of compilers and emphasize explainability and reproducibility. They typically do not provide a general-purpose canonicalization operator with explicit representation invariance guarantees, nor a dependency-specific projection contract aimed at producing the directed analysis graph used in Paper 1. Our framework serves as a compilation interface that can ingest these models into a canonical structural object suitable for uniform workload-relative dependency analysis.

3.5 Canonical graph representations, canonical labeling, and isomorphism

Canonicalization is closely related to canonical labeling and graph isomorphism. Practical toolchains (e.g., nauty/Traces) compute canonical labelings that can be used to obtain stable representatives under isomorphism for many graph families [12]. Complexity-theoretic results on graph isomorphism

Prior work	Contributes to	Gap addressed here
LLVM/MLIR	F_d, ν	no cross-domain canonical object
PDG/CFG/SSA	P_χ, U	code-specific analysis graphs
CPG	H	code-centric structural schema
PROV/workflows	$H, \mathcal{W}$	no dependency-specific projection
Graph canonicalization	κ	no domain/workload semantics

Table 1: How prior work maps to the canonical structural compiler.

Figure 2: The canonical structural object $\Sigma = (H, \mathcal{W}, \kappa)$ separates structure, workload space, and canonicalization.

also inform the limits of what a “fully” representation-invariant canonicalizer could guarantee in worst cases [1].

In software and data systems, lighter-weight canonicalization is common: stable serializations, sorting by deterministic keys, and normalization of naming/addressing conventions often suffice to erase incidental artifacts without solving full isomorphism.

Stage mapping. This body of work contributes directly to κ : it provides candidate algorithms and complexity bounds for choosing canonical representatives under a chosen equivalence. However, it does not specify which equivalence is appropriate for a domain, nor how canonicalization should interact with workload projection P_χ .

Contribution and gap relative to this framework. Canonical labeling and isomorphism contribute the mathematical grounding for strong forms of representation invariance. They do not determine which equivalences are semantically appropriate for a given domain or workload. Our framework keeps κ abstract to allow domain-appropriate choices (from stable serialization to stronger canonical labeling) while still requiring determinism and proving properties (idempotence, invariance) at the level of the compilation contract.

Table 1 summarizes how the major prior literatures align with the stages of the structural compiler.

Figure 2 summarizes the three-part decomposition of the canonical structural object, and Figure 3 summarizes the stage structure of the compiler.

Theorem Dependency Summary. The logical dependencies among the principal results are summarized in Figure 4 and listed explicitly here as a verification aid.

- Theorem 1 (Workload Preservation) depends on Definition 5 (canonical structural object) and the workload model (Section 6).

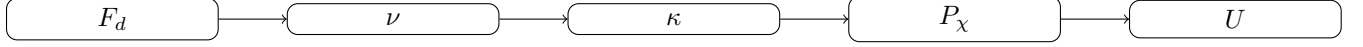


Figure 3: Compilation stages in the structural compiler $C_\chi = U \circ P_\chi \circ \kappa \circ \nu \circ F_d$.

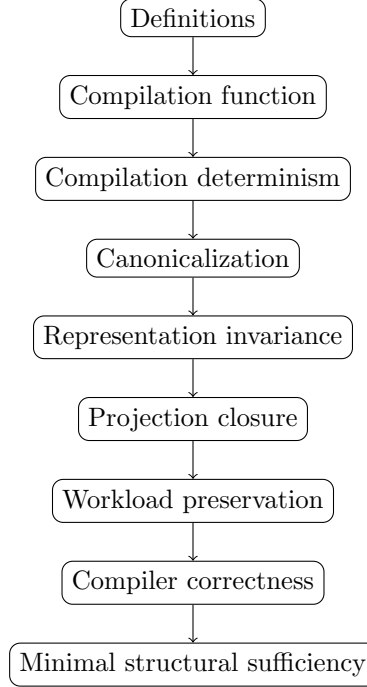


Figure 4: Dependency structure of the main theorem chain.

- Theorem 2 (Compilation Determinism) depends on Definition 7 (compilation function).
- Theorem 3 (Canonicalization Idempotence) depends on the canonical-normal-form assumption for κ .
- Theorem 4 (Projection Closure) depends on Definition 6 (counterfactual deletion) and Definition 8 (structural equivalence).
- Theorem 5 (Representation Invariance) depends on Definition 8 (structural equivalence) and the canonicalization operator κ .
- Theorem 6 (Compiler Correctness) depends on Theorems 2, 3, 4, 1, and 5.
- Theorem 7 (Minimal Structural Sufficiency) depends on Definition 6, Theorem 5, and the deletion-and-reachability dependency predicate defined in Paper 1.

4 Definitions

Notation. Let $\mathcal{D}$ denote a space of concrete system descriptions. Let $\mathcal{H}$ denote the class of finite structural graphs (Definition 4). For a fixed workload class χ , write $\mathcal{P}_\chi$ for the class of projected structural objects and $\mathcal{G}$ for the class of directed analysis graphs produced for downstream reachability-style queries.

Definition 1 (System). *A system is a finite executable structure (or process) presented via a concrete description $d \in \mathcal{D}$ such that there exists a deterministic extraction map F_d producing a finite structural graph H that is suitable for workload-relative analysis.*

Definition 2 (Component). *A component is a vertex $v \in V$ in a structural graph H representing an indivisible unit of structure at the chosen abstraction level (e.g., a function, task, resource, dataset, service, or agent).*

Definition 3 (Execution Relation). *An execution relation is a directed edge $e = (u, v) \in E$ in a structural graph H asserting that u may directly enable, invoke, produce, trigger, or otherwise precede v under the semantics of the source domain.*

Definition 4 (Structural Graph). *A structural graph is a finite typed attributed directed graph*

$$H = (V, E, \tau_V, \tau_E, \alpha),$$

where V is a finite set of components, $E \subseteq V \times V$ is a finite set of directed relations, $\tau_V : V \rightarrow \mathbb{T}_V$ and $\tau_E : E \rightarrow \mathbb{T}_E$ are typing maps, and $\alpha : (V \cup E) \rightarrow \mathbf{A}$ assigns finite structural attributes (e.g., labels, locations, or domain-specific metadata).

Definition 5 (Canonical Structural Object). *A canonical structural object is a triple*

$$\Sigma = (H, \mathcal{W}, \kappa),$$

where H is a structural graph, $\mathcal{W}$ is a workload space over H , and κ is a deterministic canonicalization operator.

Definition 6 (Counterfactual Deletion). *Let $H = (V, E, \tau_V, \tau_E, \alpha)$ be a structural graph and let $S \subseteq V$ be a designated candidate set. The counterfactual deletion of S from H is the induced structural subgraph*

$$\pi_S(H) = H \upharpoonright_{V \setminus S},$$

obtained by removing the vertices in S and all incident edges.

Definition 7 (Compilation Function). *Fix a workload class χ . A (structural) compilation function is a map*

$$C_\chi = U \circ P_\chi \circ \kappa \circ \nu \circ F_d,$$

where F_d extracts a raw structural description from $d \in \mathcal{D}$, ν normalizes, κ canonicalizes, P_χ projects to the information required by workload class χ , and U constructs the workload-specific directed graph used by later analyses.

5 Structural Graph Model

Having fixed the compilation interface (Definition 7), we now specify the structural representation extracted from concrete systems. We take the pre-workload compilation target to be the structural graph

$$H = (V, E, \tau_V, \tau_E, \alpha)$$

from Definition 4. Here V is the set of structural components, $E \subseteq V \times V$ is the set of directed structural relations, $\tau_V : V \rightarrow \mathbb{T}_V$ assigns a vertex type to each component, $\tau_E : E \rightarrow \mathbb{T}_E$ assigns an edge type to each relation, and $\alpha : (V \cup E) \rightarrow \mathbf{A}$ assigns (finite) structural attributes.

6 Workload Model

With the structural substrate fixed, we now make explicit the form of the queries (workloads) that will be posed against compiled objects. A *workload* is a query triple

$$w = (R, t, S),$$

where $R \subseteq V$ is a designated root set, $t \in V$ is a designated target, and $S \subseteq V$ is a designated candidate component set.

Theorem 1 (Workload Preservation). ***Dependencies.** Definition 5 (canonical structural object).*

Let $\Sigma = (H, \mathcal{W}, \kappa)$ be a canonical structural object (Definition 5). Let $w = (R, t, S) \in \mathcal{W}$ be a workload on H , and let $P_\chi(H)$ be the workload-class projection of H . Assume that P_χ maps the anchors R , t , and S to well-defined images R_χ , t_χ , and S_χ . Then the projected workload $w_\chi = (R_\chi, t_\chi, S_\chi)$ is well defined on $P_\chi(H)$.

Proof. Assume the hypothesis that P_χ preserves the workload anchors. Define $R_\chi := P_\chi(R)$, $t_\chi := P_\chi(t)$, and $S_\chi := P_\chi(S)$. Then (R_χ, t_χ, S_χ) is a well-formed workload triple over the projected object $P_\chi(H)$. Therefore the projected workload w_χ is well defined on $P_\chi(H)$. $\square$

7 Canonical Structural Object

Given a structural graph model, we take the principal object of study to be

$$\Sigma = (H, \mathcal{W}, \kappa),$$

where H is a structural graph, $\mathcal{W} \subseteq 2^V \times V \times 2^V$ is a workload space over H , and κ is a deterministic map that identifies a canonical representative for each structural equivalence class.

8 Compilation Function

Fix a workload class χ . The structural compiler is the composition

$$C_\chi = U \circ P_\chi \circ \kappa \circ \nu \circ F_d,$$

with stages interpreted as follows: $F_d : \mathcal{D} \rightarrow \mathcal{H}$ (domain extraction), $\nu : \mathcal{H} \rightarrow \mathcal{H}$ (normalization), $\kappa : \mathcal{H} \rightarrow \mathcal{H}$ (canonicalization), $P_\chi : (\mathcal{H} \times \mathcal{W}) \rightarrow \mathcal{P}_\chi$ (workload projection), and $U : \mathcal{P}_\chi \rightarrow \mathcal{G}$ (reachability-graph construction).

8.1 Compilation Determinism

Theorem 2 (Compilation Determinism). ***Dependencies.** Definition 7 (compilation function).*

Fix a workload class χ and let C_χ be the compilation function from Definition 7. Let $d \in \mathcal{D}$ be a concrete system description. If the stages F_d , ν , κ , P_χ , and U are deterministic, then the compiled output $C_\chi(d)$ is uniquely determined.

Proof. Assume F_d , ν , κ , P_χ , and U are deterministic. By Definition 7, C_χ is their composition. The composition of deterministic functions is deterministic, so the map $d \mapsto C_\chi(d)$ has a unique value for each input $d \in \mathcal{D}$. Therefore $C_\chi(d)$ is uniquely determined. $\square$

9 Canonicalization

Theorem 3 (Canonicalization Idempotence). ***Dependencies.** (Canonical normal form assumption for κ .)*

Let $H \in \mathcal{H}$ be a normalized structural graph. If κ maps every normalized structural object to a canonical normal form, then

$$\kappa(\kappa(H)) = \kappa(H).$$

Proof. Assume $\kappa(H)$ is a canonical normal form representative for the structural equivalence class of H . Then $\kappa(H)$ is already canonical, so re-applying κ cannot change it without violating uniqueness of the canonical representative. Therefore $\kappa(\kappa(H)) = \kappa(H)$, establishing idempotence. $\square$

10 Projection Operators

Having defined canonicalization, we now state the key compatibility property between workload projection and counterfactual deletion.

Theorem 4 (Projection Closure). ***Dependencies.** Definition 6 (counterfactual deletion); Definition 8 (structural equivalence).*

Let $H = (V, E, \tau_V, \tau_E, \alpha) \in \mathcal{H}$ be a structural graph and let $S \subseteq V$. Let P_χ denote workload-class projection. Then projection commutes with counterfactual deletion (Definition 6) up to structural equivalence (Definition 8):

$$P_\chi(\pi_S(H)) \equiv_{\text{str}} \pi_S(P_\chi(H)).$$

Proof. Assume a fixed workload class χ . By Definition 6, $\pi_S(H)$ removes exactly the vertices in S and their incident edges. The projection P_χ removes exactly the structural information irrelevant to χ . Whether we delete S first and then project, or project first and then delete the images of S , the surviving vertices and the χ -relevant relations among them coincide. Therefore the two resulting projected objects differ at most by representation artifacts (e.g., internal naming or ordering) and are structurally equivalent. Hence $P_\chi(\pi_S(H)) \equiv_{\text{str}} \pi_S(P_\chi(H))$. $\square$

11 Structural Equivalence

Later results (notably Representation Invariance, Theorem 5) use an equivalence relation that identifies structural objects differing only by representation artifacts such as naming, ordering, or serialization choices. We now make that relation explicit.

Definition 8 (Structural Equivalence). *Let $H_i = (V_i, E_i, \tau_V^i, \tau_E^i, \alpha_i)$ for $i \in \{1, 2\}$ be structural graphs. We say H_1 and H_2 are structurally equivalent, written*

$$H_1 \equiv_{\text{str}} H_2,$$

if there exists a bijection $\varphi : V_1 \rightarrow V_2$ such that:

- (i) **Edge preservation:** $(u, v) \in E_1$ iff $(\varphi(u), \varphi(v)) \in E_2$.
- (ii) **Type preservation:** for all $u \in V_1$, $\tau_V^1(u) = \tau_V^2(\varphi(u))$; and for all $(u, v) \in E_1$, $\tau_E^1(u, v) = \tau_E^2(\varphi(u), \varphi(v))$.

- (iii) **Attribute agreement up to artifacts:** for all $x \in V_1 \cup E_1$, $\alpha_1(x)$ and $\alpha_2(\varphi(x))$ agree on all semantic attributes required by the workload class under study (and may differ on representation artifacts such as internal identifiers, storage addresses, or ordering metadata).

The third condition intentionally treats “semantic” vs. “artifact” attributes as a design choice fixed by the domain and workload class; canonicalization κ is required to erase the artifact portion deterministically.

12 Representation Invariance

Theorem 5 (Representation Invariance). ***Dependencies.** Definition 8 (structural equivalence).*

Let $H_1, H_2 \in \mathcal{H}$ be normalized structural graphs. If $H_1 \equiv_{\text{str}} H_2$ (Definition 8), then $\kappa(H_1) = \kappa(H_2)$.

Proof. Assume $H_1 \equiv_{\text{str}} H_2$. By Definition 8, H_1 and H_2 agree on all semantic structure relevant to the chosen workload class and may differ only by representation artifacts. By construction, canonicalization κ deterministically erases those artifacts and selects a unique canonical representative for each structural equivalence class. Therefore $\kappa(H_1)$ and $\kappa(H_2)$ are the same representative, i.e., $\kappa(H_1) = \kappa(H_2)$. $\square$

Central guarantee. The following theorem packages the stage-level contracts into a single statement: if each stage satisfies its invariants, then dependency answers (Paper 1) are invariant under structural equivalence.

Theorem 6 (Compiler Correctness). ***Dependencies.** Definition 7; Theorems 2, 3, 4, 1, 5.*

Fix a workload class χ and let C_χ be the compilation function from Definition 7. Let $d \in \mathcal{D}$ be a concrete system description and write

$$C_\chi(d) = (G, w)$$

for the compiled analysis graph and induced workload anchors. If the compiler satisfies:

- (i) *Compilation Determinism (Theorem 2),*
- (ii) *Canonicalization Idempotence (Theorem 3),*
- (iii) *Projection Closure (Theorem 4),*
- (iv) *Workload Preservation (Theorem 1), and*
- (v) *Representation Invariance (Theorem 5),*

then every dependency query evaluated on (G, w) is representation-independent with respect to the structural equivalence relation $\equiv_{\text{str}}$.

Proof. Assume conditions (i)–(v). By Theorem 2, the pipeline output is uniquely determined for each input description d ; in particular, no stage-level nondeterminism can change the compiled analysis instance (G, w) . By Theorem 5, any two structurally equivalent normalized inputs are mapped by κ to the same canonical representative, so representation artifacts cannot change the canonical structural content that is passed downstream. By Theorem 3, once the canonical representative is reached it is stable under re-canonicalization, so repeated compilation/canonicalization cannot introduce drift. By Theorem 4, counterfactual deletion (of candidate components) and workload

projection commute up to $\equiv_{\text{str}}$, so the order in which we (i) form the workload-specific view and (ii) apply the Paper 1 deletion operation does not affect the projected structural content relevant to reachability. By Theorem 1, the workload anchors (roots, target, candidate set) are mapped consistently into the projected object on which G is constructed, so the query instance is well posed. Therefore the dependency predicate of Paper 1, when evaluated on (G, w) , depends only on canonical structural content and not on naming, ordering, or other representation artifacts. Hence dependency answers are representation-independent with respect to $\equiv_{\text{str}}$. $\square$

Theorem 7 (Minimal Structural Sufficiency). ***Dependencies.** Definition 6 (deletion); Theorem 5 (invariance).*

Let $d \in \mathcal{D}$ and let $C_\chi(d) = (G, w)$. If the compilation pipeline preserves (i) path fidelity, (ii) deletion fidelity (Definition 6), (iii) workload-anchor fidelity, and (iv) representation invariance (Theorem 5), then G contains sufficient structural information to evaluate every workload-relative dependency query in the class χ .

Proof. Assume the stated fidelity properties. Path fidelity ensures that directed reachability in G corresponds to admissible structural paths in the underlying system. Deletion fidelity (via counterfactual deletion, Definition 6) ensures that removing candidate components in the system corresponds to deleting the associated vertices/edges in G . Workload-anchor fidelity ensures that roots, targets, and candidate sets in w refer to the intended vertices in G . Finally, representation invariance ensures these correspondences do not depend on naming, ordering, or other incidental representation artifacts. Therefore G contains exactly the information needed to apply deletion-and-reachability dependency queries for workload class χ . $\square$

13 Complexity Analysis

Let d be a concrete system description and let

$$H = (V, E, \tau_V, \tau_E, \alpha)$$

be the extracted structural graph. Let

$$G_\chi = (V_\chi, E_\chi)$$

be the workload-class reachability graph produced after projection. We analyze costs in terms of

$$|d|, |V|, |E|, |V_\chi|, |E_\chi|, |R|, |S|.$$

The framework separates the cost of constructing a canonical structural object from the cost of evaluating a dependency query on a projected graph.

Stage costs. Assume adjacency-list representations for H and G_χ .

- (i) **Domain extraction** F_d : domain-dependent; at minimum $\Omega(|d|)$.
- (ii) **Normalization** ν : $O(|V| + |E|)$.
- (iii) **Canonicalization** κ : write $C_\kappa(H)$ for the canonicalization cost on H . A common deterministic strategy (e.g., sorting vertices/edges for stable serialization) is

$$O((|V| + |E|) \log(|V| + |E|)),$$

while stronger forms (e.g., canonical labeling up to isomorphism) may be more expensive in the worst case and are intentionally not collapsed into a single polynomial claim.

- (iv) **Workload projection** P_χ : $O(|V| + |E|)$.
- (v) **Reachability-graph construction** U : $O(|V_\chi| + |E_\chi|)$.

Query costs on G_χ . For a workload $w_\chi = (R_\chi, t_\chi, S_\chi)$ on G_χ :

- (i) **Deletion** π_S : $O(|V_\chi| + |E_\chi|)$, e.g., by filtering to active vertices and incident edges.
- (ii) **Reachability** ρ : $O(|V_\chi| + |E_\chi|)$ via BFS/DFS.

Consequently, the total cost of a single dependency query after compilation is

$$O(|V_\chi| + |E_\chi|).$$

End-to-end bound. A conservative end-to-end bound for producing G_χ from d and then answering one dependency query is

$$O(|d| + C_\kappa(H) + |V| + |E| + |V_\chi| + |E_\chi|).$$

14 Discussion

The technical development above defines an interface between two kinds of objects: (i) heterogeneous, domain-specific system descriptions, and (ii) workload-relative predicates (Paper 1) that require a directed analysis graph and a consistent notion of counterfactual deletion. Making that interface explicit has several implications.

What the framework enables. Once a domain admits a front end F_d into a typed structural graph H , later analyses can be expressed as workload-class projections and graph queries on $G_\chi = U(P_\chi(\kappa(\nu(F_d(d)))))$. This includes, beyond workload-relative dependency (Paper 1):

- (i) **Resilience analysis:** quantify how dependency answers change under removal of components or relations, and identify minimal cut sets or “single points of failure” relative to a workload class.
- (ii) **Redundancy analysis:** detect multiple disjoint structural witnesses for reachability and characterize when dependency is robust to counterfactual deletions.
- (iii) **Counterfactual analysis:** compare worlds produced by families of deletions (or hypothetical additions), using κ to factor out representational variation.
- (iv) **Optimization and refactoring:** treat κ and P_χ as contracts that allow refactorings or compilation changes while preserving workload answers.
- (v) **Governance and accountability:** when systems are socio-technical, canonical structural objects provide a stable substrate for auditing and policy reasoning (e.g., continuity-oriented governance layers).

Interpretation of canonicalization. Canonicalization κ should not be interpreted as claiming a single universal representation across domains; different domains may instantiate κ differently while preserving the same compiler contract. Rather, it enforces a deterministic choice of representative within a domain- and workload-appropriate structural equivalence class (Definition 8). This shifts debates about “the right graph” into explicit modeling choices: which attributes are semantic for a workload class, and which are artifacts that must not influence answers.

Relationship to existing IR and provenance tooling. Because the framework is staged, it can be implemented using existing compiler IR infrastructure (for software) [10, 11] or provenance/workflow graph infrastructure (for data systems) [7, 5], while still enforcing the invariants required for representation-independent downstream queries.

15 Limitations

Equivalence is workload- and domain-relative. Structural equivalence (Definition 8) depends on what counts as a semantic attribute for a given workload class. Different choices can change both the strength of representation invariance and the feasibility/cost of κ .

Canonicalization may be expensive in the worst case. While many practical settings admit light-weight deterministic canonicalization (e.g., stable serialization), stronger invariance goals can approach canonical labeling complexity. The framework isolates this cost (Section 13) but does not remove it.

Front-end assumptions. The framework assumes a deterministic extraction function F_d . In practice, extraction may depend on dynamic execution, environment configuration, or incomplete specifications; such settings require either enriched system descriptions or probabilistic/parametric structural objects.

Projection can be lossy. Workload projection P_χ is intentionally selective. If a later analysis needs information not preserved by the chosen projection, the projection contract must be strengthened (or multiple projections must coexist).

16 Future Work

Several extensions are natural.

- (i) **Domain instantiations and case studies:** define concrete F_d, ν, κ, P_χ for at least one compiler domain (e.g., LLVM/MLIR) and one provenance/workflow domain (e.g., W3C PROV-style graphs), and empirically evaluate invariance and cost.
- (ii) **Richer structural equivalences:** study families of $\equiv_{\text{str}}$ that incorporate attribute abstractions, partial observability, or semantic equivalences stronger than graph isomorphism.
- (iii) **Incremental and streaming compilation:** support evolving systems by maintaining $\kappa(H)$ and G_χ under updates, enabling continuous dependency and resilience monitoring.
- (iv) **Query languages and tool support:** expose workload classes and projections through a declarative interface so that analyses can be authored without hard-coding domain-specific graph transformations.
- (v) **Governance layers:** connect canonical structural objects to policy and continuity governance artifacts to make accountability constraints first-class alongside technical dependency queries.

17 Conclusion

Paper 1 formalized workload-relative dependency as a predicate computed by counterfactual deletion and reachability on a directed analysis graph. This paper addressed the missing prior question: given a heterogeneous concrete system description, what exactly is the graph on which that predicate operates?

We answered by defining a deterministic structural compilation contract $C_\chi = U \circ P_\chi \circ \kappa \circ \nu \circ F_d$ that produces a workload-class analysis graph (and consistent workload anchors) in a representation-independent way. Together, Papers 1 and 2 establish a mathematical foundation for workload-relative structural analysis in which downstream results depend on canonical structural content rather than incidental representation artifacts. In particular, Theorem 6 establishes representation-independent dependency answers for compiled instances, and Theorem 7 establishes that the compiled analysis graph contains sufficient information to evaluate dependency queries in the workload class.

With the dependency predicate and its compilation target fixed, subsequent papers can focus on analyses enabled by this foundation—including dependency analysis at scale (Paper 1), and extensions such as resilience, redundancy, counterfactual, optimization, and governance-oriented reasoning built on top of canonical structural objects.

A Notation (alphabetical)

Symbol	Meaning
C_χ	Structural compilation function for workload class χ (Definition 7)
$d \in \mathcal{D}$	Concrete system description / description space
E	Edge set of a structural graph H
F_d	Domain extraction / front end (Section 8)
G_χ	Workload-class directed analysis graph produced after projection
H	Structural graph (Definition 4)
P_χ	Workload-class projection operator (Section 8)
R, t, S	Workload anchors: roots, target, candidate set (Section 6)
U	Reachability-graph construction stage (Section 8)
V	Vertex set (components) of a structural graph H
$w = (R, t, S)$	Workload/query triple (Section 6)
α	Attribute map of a structural graph H (Definition 4)
$\equiv_{\text{str}}$	Structural equivalence relation (Definition 8)
κ	Canonicalization operator (Definition 5)
$\mathcal{W}$	Workload space over H (Definition 5)
ν	Normalization stage (Section 8)
$\pi_S(H)$	Counterfactual deletion of S from H (Definition 6)
Σ	Canonical structural object $(H, \mathcal{W}, \kappa)$ (Definition 5)
τ_V, τ_E	Vertex/edge typing maps of H (Definition 4)

Table 2: Core notation used throughout the manuscript, alphabetized by symbol.

B Appendix A

C Appendix B

References

- [1] László Babai. Graph isomorphism in quasipolynomial time. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 684–697, 2016.
- [2] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. *Proceedings of the 8th International Conference on Database Theory (ICDT)*, pages 316–330, 2001.
- [3] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.
- [4] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 25–35, 1991.
- [5] Ewa Deelman, Dennis Gannon, Matthew Shields, and Ian Taylor. Workflows and e-science: An overview of workflow system features and capabilities. *Future Generation Computer Systems*, 25(5):528–540, 2009.
- [6] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems*, 9(3):319–349, 1987.
- [7] Paul Groth and Luc Moreau. PROV-Overview: An overview of the PROV family of documents. W3c recommendation, W3C, April 2013.
- [8] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Stefan Schlobach, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):71:1–71:37, 2021.
- [9] Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems*, 1(1):121–141, 1979.
- [10] LLVM Project. Llvm language reference manual. Online documentation.
- [11] LLVM Project. Mlir: Multi-level intermediate representation. Online documentation.
- [12] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, ii. *Journal of Symbolic Computation*, 60:94–112, 2014.
- [13] Steven S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
- [14] Mark Weiser. Program slicing. In *Proceedings of the 5th International Conference on Software Engineering (ICSE)*, pages 439–449, 1981.

- [15] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE Symposium on Security and Privacy*, pages 590–604. IEEE, 2014.